

README

Project Title

Schema Scence: Query Aware Optimization for RDB-to-NoSQL Transformation

Project Overview

This project implements a query-aware framework for RDB-to-NoSQL schema evaluation and migration planning. It follows the QBMetrics and Metamorfose methodology and uses Directed Acyclic Graphs (DAGs) to represent both candidate NoSQL schemas and SQL query workloads.[file:318]

The workflow of the project includes four major stages: definition of candidate schemas and queries, calculation of metrics and scores, schema evaluation and ranking, and Python-based migration planning for NoSQL document generation.[file:318]

The project also provides a Streamlit dashboard to visualize schemas, inspect metrics, compare scenarios, analyze DAG structures, and preview migration output.[file:318]

Software Required

The following software is required to run this project:

- Python 3.11
- pip
- Streamlit
- Required Python libraries from `requirements.txt`

This project is executed directly in the system Python environment and does not use a virtual environment.[file:318]

Install Required Packages

Open a terminal in the project folder and run:

```
pip install -r requirements.txt
```

If any package is still missing, install it manually using `pip`.

Project Directory Structure

The project directory is organized as follows:

```
project/
├── data/
│   ├── dvdstore_setup.sql
│   ├── sample_data.sql
│   └── setup_db.sql
├── experiments/
│   ├── reproduce_paper.py
│   ├── scenario1.py
│   ├── scenario2.py
│   └── scenario3.py
├── modules/
│   ├── __init__.py
│   ├── dag_builder.py
│   ├── dvdstore_adapter.py
│   ├── metamorfose.py
│   ├── metrics.py
│   ├── models.py
│   ├── query_guidelines.py
│   ├── reports.py
│   ├── schema_generator.py
│   ├── schema_library.py
│   └── scores.py
├── results/
├── ui/
│   └── app.py
├── config.py
├── requirements.txt
└── run_all.py
```

Explanation of File Directory

data/

This folder contains the SQL files required to initialize and populate the relational source database used in the project.

- `dvdstore_setup.sql` -- creates the DVD Store schema.
- `sample_data.sql` -- inserts sample records into the database.
- `setup_db.sql` -- helps prepare the database before evaluation and migration.

These files are used as the relational input side of the RDB-to-NoSQL pipeline.[file:318]

experiments/

This folder contains scripts for reproducing the experimental evaluation presented in the project.

- `reproduce_paper.py` -- reproduces the overall experiment workflow.
- `scenario1.py` -- runs Scenario 1, which ranks schemas by minimum number of required collections.
- `scenario2.py` -- runs Scenario 2, which ranks schemas based on access-pattern matching.
- `scenario3.py` -- runs Scenario 3, which ranks schemas with preferred-query weighting.
[file:318]

These scripts correspond to the scenario-based evaluation described in the implementation.
[file:318]

modules/

This folder contains the main backend logic of the project.

- `__init__.py` -- marks the folder as a Python module.
- `dag_builder.py` -- builds DAG representations of schemas and queries.[file:318]
- `dvdstore_adapter.py` -- connects the DVD Store relational schema to the project pipeline.
- `metamorphose.py` -- implements the migration-planning logic inspired by the Metamorphose framework.[file:318]
- `metrics.py` -- computes metrics such as Path, SubPath, IndPath, DirEdge, AllEdge, ReqColls, and FArray.[file:318]
- `models.py` -- defines internal data structures used across the system.
- `query_guidelines.py` -- stores or supports rules related to query interpretation and transformation.
- `reports.py` -- generates result summaries and reporting outputs.
- `schema_generator.py` -- generates candidate NoSQL schemas.
- `schema_library.py` -- stores or manages schema definitions for S_A , S_B , S_C , and S_D .
- `scores.py` -- calculates QScore and SScore for schema ranking.[file:318]

This folder is the core implementation layer of the project.[file:318]

results/

This folder stores generated outputs such as metric tables, rankings, charts, reports, and other result files produced by experiments or the dashboard.

ui/

This folder contains the Streamlit user interface.

- `app.py` -- main dashboard application file.

This is the main entry point for launching the dashboard interface of the project.[file:318]

`config.py`

This file stores configurable project settings such as paths, parameters, or scenario-related values used during execution.

`requirements.txt`

This file contains the list of Python packages required to run the project.

`run_all.py`

This file serves as a master script to execute the full workflow or multiple parts of the system together.

How to Run the Project

Step 1: Open terminal in the project folder

```
cd path/to/your/project
```

Step 2: Check Python version

```
python --version
```

Expected version:

```
Python 3.11.x
```

Step 3: Install dependencies

```
pip install -r requirements.txt
```

Step 4: Run the Streamlit dashboard

Since the main UI file is located in the `ui` folder, run:

```
streamlit run ui/app.py
```

Step 5: Open the browser

After running the above command, Streamlit will show a local URL such as:

```
http://localhost:8501
```

Open it in the browser to access the dashboard.

Running Experiment Scripts

If you want to run the evaluation scripts directly instead of opening the dashboard, use the following commands:

```
python experiments/reproduce_paper.py
python experiments/scenario1.py
python experiments/scenario2.py
python experiments/scenario3.py
```

These scripts execute the experimental scenarios discussed in the project.[file:318]

Running the Full Workflow

To run the overall integrated workflow, use:

```
python run_all.py
```

This is useful when you want to execute the complete pipeline from schema/query processing to ranking and result generation.

What the Project Produces

When the project runs successfully, it can produce:

- schema DAG visualizations,
- query DAG representations,
- metric tables,
- schema ranking outputs,
- scenario-wise evaluation summaries,
- migration planning outputs,

- JSON-style NoSQL transformation previews, and
- dashboard-based interactive analysis views.[file:318]

Typical Command Sequence

Use this exact sequence:

```
cd path/to/your/project
python --version
pip install -r requirements.txt
streamlit run ui/app.py
```

For scenario scripts:

```
python experiments/scenario1.py
python experiments/scenario2.py
python experiments/scenario3.py
```

For complete execution:

```
python run_all.py
```

Notes

- Use Python 3.11.
- Do not create a virtual environment for this setup.
- Install packages directly using `pip install -r requirements.txt`.
- Run the dashboard from `ui/app.py`.
- Run scenario files separately from the `experiments/` folder when needed.

Conclusion

This project is a Python 3.11 based implementation of a QBMetrics-driven RDB-to-NoSQL schema evaluation, ranking, and migration-planning system. The codebase is organized into data preparation, experimental evaluation, backend modules, dashboard UI, and full-pipeline execution scripts, making it easy to run individual scenarios or the complete workflow.

[file:318]